

รายงานการปรับปรุงประสิทธิภาพ Retrieval Performance Optimization

ลด latency ของ retrieval แบบ behavior-preserving
พร้อมการเปรียบเทียบ old-vs-new และ CPU/GPU

CyberCase Intelligence Framework — branch perf/retrieval-batch-expand
วันที่ 11 กรกฎาคม พ.ศ. 2569 | การทดสอบ retrieval-only (ไม่ใช่ LLM)

1. ที่มา (ทำไมต้องแก้)

จากการวัด latency พบว่า retrieval กินเวลาส่วนใหญ่ และตัวถ่วงหลักคือ graph expansion บน Neo4j ที่วนขยายทีละ seed: retrieve_multi_quota() แยกสำนวนเป็น N sub-query แล้ววนทีละตัว □ แต่ละตัวเรียก expand() ที่วน _expand_single() ทีละ seed โดยแต่ละ seed เปิด Neo4j session ใหม่ + ยิง 3 Cypher (center + outgoing + incoming)

ต่อ 1 สำนวน (~7 sub-query × ~5 seed × 3 query) = ~100 sequential round-trip ไป cloud Neo4j — คอขวดคือจำนวน round-trip ไม่ใช่การคำนวณ

2. สิ่งที่ทำ (3 อย่าง)

#	สิ่งที่ทำ	สถานะ	ผล
1	Batch Neo4j expansion — ยิง 3 Cypher (UNWIND) รวมทั้ง seed list แทน 3×N	□ commit 0e91f9c	เร็วขึ้น 3.5× ที่ขั้น expand ผลเหมือนเดิม
2/3	Batch ข้าม sub-query — embed/Qdrant/rerank/graph ครึ่งเดียว	□ ทิ้ง	GPU ~เท่าเดิม, CPU +10% ไม่สม่ำเสมอ; batch rerank ยิ่งช้า
4	Device-aware loading — auto GPU/CPU + fp16 เฉพาะ GPU + log + override	□ commit 697731b	แก้ fp16-on-CPU □ CPU เร็วขึ้น ~20%

3. Item 1 — Batched Neo4j Expansion

เดิม: N seed × 3 Cypher × session ใหม่ = 3N round-trip □ ใหม่: 3 Cypher รวม (UNWIND \$ids) ใน session เดียว = 3 round-trip

เทียบ old vs new บน input เดียวกัน (GPU, controlled):

sample	old (per-seed loop)	new (batched UNWIND)	speedup
inc_auto_001 (8 seeds)	1971 ms	543 ms	3.6×
inc_auto_002 (8 seeds)	1226 ms	378 ms	3.2×
inc_auto_003 (8 seeds)	959 ms	222 ms	4.3×
inc_auto_004 (8 seeds)	1303 ms	427 ms	3.0×
รวม	5458 ms	1571 ms	3.5×

ความถูกต้อง: SET EQUIVALENCE PASS — center nodes (ลำดับเดียวกัน) + neighbor/edge sets ทุก subgraph เหมือนเดิมเป๊ะ (set ของ ID ที่ retrieve ได้ไม่เปลี่ยน)

4. Item 2/3 — Batch ข้าม sub-query (ทิ้ง)

หลัง item 1 profile ต่อ sub-query สมดุลกัน 3 ส่วน (GPU): vector 0.50s / rerank 0.52s / graph 0.50s จึงลอง batch ทั้ง 3 ส่วน ผล (set เหมือนเดิมทุกกรณี แต่ latency ไม่คุ้ม):

ฮาร์ดแวร์	วิธี	ผล	สรุป
GPU	batched vs seq (แก้ order effect)	14.5s vs 14.7s = 1.01×	เท่าเดิม

ฮาร์ดแวร์	วิธี	ผล	สรุป
CPU	batched (รวม rerank) vs seq	210.4s vs 186.4s = 0.89×	ช้ากว่า
CPU	hybrid (batch เวิร์น rerank) vs seq	170.3s vs 187.2s = 1.10×	ดีขึ้นเล็กน้อย ไม่สม่ำเสมอ

สาเหตุ: การ batch cross-encoder rerank (รวม pair ทุก sub-query เป็น predict เดียว) ช้ากว่าการ rerank ทีละ query ทั้งบน GPU (VRAM pressure) และ CPU — stage timing CPU: search_all_batch=13.7s, rerank_batch=69.8s, expand_batch=1.3s

ตัดสินใจ: ทั้ง item 2/3 — ได้ ~10% บน CPU ไม่สม่ำเสมอ แลกกับโค้ดซับซ้อน + float-equivalence risk ไม่คุ้ม

Insight: คอขวดจริงบน production (CPU) คือ cross-encoder reranker เอง (~7-8s/sub-query) ที่ batch ไม่ได้ = งานคนละก้อน

5. Item 4 — Device-Aware Loading

ปัญหา: USE_FP16 ถูก hardcode = True แต่ fp16 เป็น GPU optimization บน CPU ต้อง emulate (ช้า) และ production (Railway) รันบน CPU (torch CPU wheel + python:3.11-slim)

แก้: config.DEVICE = cuda ถ้ามี GPU ไม่งั้น cpu; USE_FP16 = (DEVICE==cuda); RAG_DEVICE=cpu|cuda override; log ตอน startup; reranker ส่ง device= ชัดเจน

ยืนยัน: default □ cuda (fp16=True) reranker บน cuda:0; RAG_DEVICE=cpu □ cpu (fp16=False)

6. เปรียบเทียบ CPU / GPU (โค้ดสุดท้าย: item 1 + device fix)

e2e retrieval (production sequential path):

sample (sub-q)	GPU (cuda, fp16)	CPU fp32 (prod, ใหม่)	CPU fp16 (เก่า)
inc_auto_001 (9)	16.8s	51.6s	~68–76s
inc_auto_002 (8)	13.5s	55.2s	~67–70s
inc_auto_003 (6)	10.1s	39.4s	~41–50s

อ่านตาราง: GPU เร็วกว่า CPU ~3-4× แต่ production คือ CPU (Railway ไม่มี GPU); CPU fp32 (ใหม่) เร็วกว่า CPU fp16 (เก่า) ~20% = ผลของการแก้ device; latency แปรผันตาม จำนวน sub-query + cloud variance สูง (ตัวเลข e2e เป็นแนวโน้ม ไม่ใช่ค่าตายตัว)

□□ ตัวเลข GPU วัดบน RTX 2050 ในเครื่อง dev — production ใช้ CPU เสมอ การตัดสินใจ latency เชิง production ต้องอิงคอลัมน์ CPU fp32

7. สรุปการทดสอบ (ครบทุกแบบ + บันทึกเวลาหุ้รอบ)

การวัด	GPU	CPU
item 1 equivalence (controlled)	□	network-bound
stage profile ต่อ sub-query	□	—
item 2/3 batched vs seq	□	□
item 2/3 hybrid vs seq	—	□
โค้ดสุดท้าย e2e	□	□ (fp32)
device detection	□	□

ทุกการทดสอบเป็น retrieval-only ไม่ใช้ LLM (ไม่มีค่า API)

8. สถานะ Push / Merge

อยู่บน main แล้ว: PR #12 (single-call C), PR #13 (fix follow-up empty answer)

รอ push + PR — branch perf/retrieval-batch-expand (3 commits):

- 0e91f9c perf(retrieval): batch Neo4j graph expansion (item 1)
- 697731b feat(retrieval): device-aware model loading (item 4)
- 06a767a docs(retrieval): perf optimization summary

ทั้งหมด behavior-preserving + verified GPU/CPU □ พร้อม merge

ทิ้งแล้ว: item 2/3 (revert ออก ไม่ commit)

9. งานต่อยอด (นอกขอบเขต)

- คอขวดจริง = reranker บน CPU — ตัวเลือก: ลดจำนวน sub-query, reranker เล็ก/เร็วกว่า, rerank ครั้งเดียวบน merged candidates, หรือย้าย rag_service ไป GPU host
- retrieval quality: described-cue coverage 0.42 (คอขวดคุณภาพ แยกจาก latency)